

UNIT37: GLTF MODEL - ANIMATIONS

【学習要項】

- ☐ Animations
- ☐ Skins
- ☐ Joints

【演習手順】

1. アニメーションの取得

- ① gltf_model クラスに skin 構造体と animation 構造体を定義する

```
1: struct skin
2: {
3:     std::vector<DirectX::XMFLOAT4X4> inverse_bind_matrices;
4:     std::vector<int> joints;
5: };
6: std::vector<skin> skins;
7:
8: struct animation
9: {
10:     std::string name;
11:     float duration{ 0.0f };
12:     struct channel
13:     {
14:         int sampler{ -1 };
15:         int target_node{ -1 };
16:         std::string target_path;
17:     };
18:     std::vector<channel> channels;
19:
20:     struct sampler
21:     {
22:         int input{ -1 };
23:         int output{ -1 };
24:         std::string interpolation;
25:     };
26:     std::vector<sampler> samplers;
27:
28:     std::unordered_map<int/*sampler.input*/, std::vector<float>> timelines;
29:     std::unordered_map<int/*sampler.output*/, std::vector<DirectX::XMFLOAT3>> scales;
30:     std::unordered_map<int/*sampler.output*/, std::vector<DirectX::XMFLOAT4>> rotations;
31:     std::unordered_map<int/*sampler.output*/, std::vector<DirectX::XMFLOAT3>> translations;
32: };
33: std::vector<animation> animations;
```

- ② gltf_model クラスに fetch_animations メンバ関数を実装する

```
1: void gltf_model::fetch_animations(const tinygltf::Model& gltf_model)
2: {
3:     using namespace std;
4:     using namespace tinygltf;
5:     using namespace DirectX;
6:
7:     for (vector<Skin>::const_reference transmission_skin : gltf_model.skins)
8:     {
9:         skin& skin{ skins.emplace_back() };
10:         const Accessor& gltf_accessor{ gltf_model.accessors.at(transmission_skin.inverseBindMatrices) };
11:         const BufferView& gltf_buffer_view{ gltf_model.bufferViews.at(gltf_accessor.bufferView) };
12:         skin.inverse_bind_matrices.resize(gltf_accessor.count);
13:         memcpy(skin.inverse_bind_matrices.data(), gltf_model.buffers.at(gltf_buffer_view.buffer).data.data() +
14:             gltf_buffer_view.byteOffset + gltf_accessor.byteOffset, gltf_accessor.count * sizeof(XMFLOAT4X4));
15:         skin.joints = transmission_skin.joints;
16:     }
17:
18:     for (vector<Animation>::const_reference gltf_animation : gltf_model.animations)
19:     {
20:         animation& animation{ animations.emplace_back() };
```

UNIT37: GLTF MODEL - ANIMATIONS

```
21:   animation.name = gltf_animation.name;
22:   for (vector<AnimationSampler>::const_reference gltf_sampler : gltf_animation.samplers)
23:   {
24:       animation::sampler& sampler{ animation.samplers.emplace_back() };
25:       sampler.input = gltf_sampler.input;
26:       sampler.output = gltf_sampler.output;
27:       sampler.interpolation = gltf_sampler.interpolation;
28:
29:       const Accessor& gltf_accessor{ gltf_model.accessors.at(gltf_sampler.input) };
30:       const BufferView& gltf_buffer_view{ gltf_model.bufferViews.at(gltf_accessor.bufferView) };
31:       const pair<unordered_map<int, vector<float>>::iterator, bool>& timelines{
32:           animation.timelines.emplace(gltf_sampler.input, gltf_accessor.count) };
33:       if (timelines.second)
34:       {
35:           memcpy(timelines.first->second.data(), gltf_model.buffers.at(gltf_buffer_view.buffer).data.data() +
36:               gltf_buffer_view.byteOffset + gltf_accessor.byteOffset, gltf_accessor.count * sizeof(FLOAT));
37:       }
38:   }
39:   for (vector<AnimationChannel>::const_reference gltf_channel : gltf_animation.channels)
40:   {
41:       animation::channel& channel{ animation.channels.emplace_back() };
42:       channel.sampler = gltf_channel.sampler;
43:       channel.target_node = gltf_channel.target_node;
44:       channel.target_path = gltf_channel.target_path;
45:
46:       const AnimationSampler& gltf_sampler{ gltf_animation.samplers.at(gltf_channel.sampler) };
47:       const Accessor& gltf_accessor{ gltf_model.accessors.at(gltf_sampler.output) };
48:       const BufferView& gltf_buffer_view{ gltf_model.bufferViews.at(gltf_accessor.bufferView) };
49:       if (gltf_channel.target_path == "scale")
50:       {
51:           const pair<unordered_map<int, vector<XMFLAT3>>::iterator, bool>& scales{
52:               animation.scales.emplace(gltf_sampler.output, gltf_accessor.count) };
53:           if (scales.second)
54:           {
55:               memcpy(scales.first->second.data(), gltf_model.buffers.at(gltf_buffer_view.buffer).data.data() +
56:                   gltf_buffer_view.byteOffset + gltf_accessor.byteOffset, gltf_accessor.count * sizeof(XMFLAT3));
57:           }
58:       }
59:       else if (gltf_channel.target_path == "rotation")
60:       {
61:           const pair<unordered_map<int, vector<XMFLAT4>>::iterator, bool>& rotations{
62:               animation.rotations.emplace(gltf_sampler.output, gltf_accessor.count) };
63:           if (rotations.second)
64:           {
65:               memcpy(rotations.first->second.data(), gltf_model.buffers.at(gltf_buffer_view.buffer).data.data() +
66:                   gltf_buffer_view.byteOffset + gltf_accessor.byteOffset, gltf_accessor.count * sizeof(XMFLAT4));
67:           }
68:       }
69:       else if (gltf_channel.target_path == "translation")
70:       {
71:           const pair<unordered_map<int, vector<XMFLAT3>>::iterator, bool>& translations{
72:               animation.translations.emplace(gltf_sampler.output, gltf_accessor.count) };
73:           if (translations.second)
74:           {
75:               memcpy(translations.first->second.data(), gltf_model.buffers.at(gltf_buffer_view.buffer).data.data() +
76:                   gltf_buffer_view.byteOffset + gltf_accessor.byteOffset, gltf_accessor.count * sizeof(XMFLAT3));
77:           }
78:       }
79:   }
80: }
81: // Find a longest animation duration in timeline of each channel.
82: for (decltype/animations)::reference animation : animations)
83: {
84:     for (decltype(animation.timelines)::reference timelines : animation.timelines)
```

UNIT37: GLTF MODEL - ANIMATIONS

```
85:     {
86:         animation.duration = std::max<float>(animation.duration, timelines.second.back());
87:     }
88: }
89: }
```

③ gltf_model クラスのコンストラクタから fetch_animation メンバ関数を呼び出す

④ gltf_model クラスにボーン行列の構造体と定数バッファを定義する

```
1: static const size_t PRIMITIVE_MAX_JOINTS = 512;
2: struct primitive_joint_constants
3: {
4:     DirectX::XMFLOAT4x4 matrices[PRIMITIVE_MAX_JOINTS];
5: };
6: Microsoft::WRL::ComPtr<ID3D11Buffer> primitive_joint_cbuffer;
```

⑤ gltf_model クラスのコンストラクタでボーン行列の定数バッファを生成する

```
1: // D3D11_BUFFER_DESC buffer_desc{};
2: buffer_desc.ByteWidth = sizeof(primitive_joint_constants);
3: buffer_desc.Usage = D3D11_USAGE_DEFAULT;
4: buffer_desc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
5: hr = device->CreateBuffer(&buffer_desc, NULL, primitive_joint_cbuffer.ReleaseAndGetAddressOf());
6: _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
```

⑥ gltf_model クラスの render メンバ関数の引数を追加する

```
const std::vector<node>& animated_nodes
```

⑦ gltf_model クラスの render メンバ関数の先頭に下記コードを追加する

```
const std::vector<node>& nodes{ animated_nodes.size() > 0 ? animated_nodes : gltf_model::nodes };
```

⑧ gltf_model クラスの render メンバ関数の traverse ラムダ式内に下記コードを追加する

```
1: if (node.skin > -1)
2: {
3:     const skin& skin{ skins.at(node.skin) };
4:     primitive_joint_constants primitive_joint_data{};
5:     for (size_t joint_index = 0; joint_index < skin.joints.size(); ++joint_index)
6:     {
7:         XMStoreFloat4x4(&primitive_joint_data.matrices[joint_index],
8:             XMLoadFloat4x4(&skin.inverse_bind_matrices.at(joint_index)) *
9:             XMLoadFloat4x4(&nodes.at(skin.joints.at(joint_index)).global_transform) *
10:            XMMatrixInverse(NULL, XMLoadFloat4x4(&node.global_transform))
11:        );
12:    }
13:    immediate_context->UpdateSubresource(primitive_joint_cbuffer.Get(), 0, 0, &primitive_joint_data, 0, 0);
14:    immediate_context->VSSetConstantBuffers(2, 1, primitive_joint_cbuffer.GetAddressOf());
15: }
```

⑨ gltf_model クラスに animate メンバ関数を実装する

```
1: void gltf_model::animate(size_t animation_index, float time, std::vector<node>& animated_nodes)
2: {
3:     using namespace std;
4:     using namespace DirectX;
5:
6:     function<size_t(const vector<float>&, float, float&)> indexof{
7:         [](const vector<float>& timelines, float time, float& interpolation_factor)->size_t {
8:             const size_t keyframe_count{ timelines.size() };
9:             if (time > timelines.at(keyframe_count - 1))
10:             {
11:                 interpolation_factor = 1.0f;
```

```
12:     return keyframe_count - 2;
13: }
14: else if (time < timelines.at(0))
15: {
16:     interpolation_factor = 0.0f;
17:     return 0;
18: }
19: size_t keyframe_index{ 0 };
20: for (size_t time_index = 1; time_index < keyframe_count; ++time_index)
21: {
22:     if (time < timelines.at(time_index))
23:     {
24:         keyframe_index = max<size_t>(0LL, time_index - 1);
25:         break;
26:     }
27: }
28: interpolation_factor = (time - timelines.at(keyframe_index + 0)) /
29:     (timelines.at(keyframe_index + 1) - timelines.at(keyframe_index + 0));
30: return keyframe_index;
31: } };
```

```
32:
33: if (animations.size() > 0)
34: {
35:     const animation& animation{ animations.at(animation_index) };
36:     for (vector<animation::channel>::const_reference channel : animation.channels)
37:     {
38:         const animation::sampler& sampler{ animation.samplers.at(channel.sampler) };
39:         const vector<float>& timeline{ animation.timelines.at(sampler.input) };
40:         if (timeline.size() == 0)
41:         {
42:             continue;
43:         }
44:         float interpolation_factor{};
45:         size_t keyframe_index{ indexof(timeline, time, interpolation_factor) };
46:         if (channel.target_path == "scale")
47:         {
48:             const vector<XMFloat3>& scales{ animation.scales.at(sampler.output) };
49:             XMStoreFloat3(&animated_nodes.at(channel.target_node).scale,
50:                 XMVectorLerp(XMLoadFloat3(&scales.at(keyframe_index + 0)),
51:                     XMLoadFloat3(&scales.at(keyframe_index + 1)), interpolation_factor));
52:         }
53:         else if (channel.target_path == "rotation")
54:         {
55:             const vector<XMFloat4>& rotations{ animation.rotations.at(sampler.output) };
56:             XMStoreFloat4(&animated_nodes.at(channel.target_node).rotation,
57:                 XMQuaternionNormalize(XMQuaternionSlerp(XMLoadFloat4(&rotations.at(keyframe_index + 0)),
58:                     XMLoadFloat4(&rotations.at(keyframe_index + 1)), interpolation_factor)));
59:         }
60:         else if (channel.target_path == "translation")
61:         {
62:             const vector<XMFloat3>& translations{ animation.translations.at(sampler.output) };
63:             XMStoreFloat3(&animated_nodes.at(channel.target_node).translation,
64:                 XMVectorLerp(XMLoadFloat3(&translations.at(keyframe_index + 0)),
65:                     XMLoadFloat3(&translations.at(keyframe_index + 1)), interpolation_factor));
66:         }
67:     }
68:     cumulate_transforms(animated_nodes);
69: }
70: }
```

2. シェーダーの実装 (変更)

① HLSL ヘッダファイル(gltf_model.hlsl)に PRIMITIVE_JOINT_CONSTANTS 定数バッファを定義する

```
1: static const uint PRIMITIVE_MAX_JOINTS = 512;
2: cbuffer PRIMITIVE_JOINT_CONSTANTS : register(b2)
```

```
3: {
4:   row_major float4x4 joint_matrices[PRIMITIVE_MAX_JOINTS];
5: };
```

②頂点シェーダー(gltf_model_ps.hlsl)の main 関数を変更する

```
1: VS_OUT main(VS_IN vin)
2: {
3:   float sigma = vin.tangent.w;
4:
5:   if (skin > -1)
6:   {
7:     row_major float4x4 skin_matrix =
8:       vin.weights.x * joint_matrices[vin.joints.x] +
9:       vin.weights.y * joint_matrices[vin.joints.y] +
10:      vin.weights.z * joint_matrices[vin.joints.z] +
11:      vin.weights.w * joint_matrices[vin.joints.w];
12:     vin.position = mul(float4(vin.position.xyz, 1), skin_matrix);
13:     vin.normal = normalize(mul(float4(vin.normal.xyz, 0), skin_matrix));
14:     vin.tangent = normalize(mul(float4(vin.tangent.xyz, 0), skin_matrix));
15:   }
16:
17:   VS_OUT vout;
18:   vin.position.w = 1;
19:   vout.position = mul(vin.position, mul(world, view_projection));
20:   vout.w_position = mul(vin.position, world);
21:   vin.normal.w = 0;
22:   vout.w_normal = normalize(mul(vin.normal, world));
23:   vin.tangent.w = 0;
24:   vout.w_tangent = normalize(mul(vin.tangent, world));
25:   vout.w_tangent.w = sigma;
26:   vout.texcoord = vin.texcoord;
27:   return vout;
28: }
```

3. gltf_model クラスの render メンバ関数を呼び出し、モデルが描画されることを確認する
※モデルデータを「CesiumMan」または「BrainStem」に変更する

【例】

```
static std::vector<gltf_model::node> animated_nodes{ gltf_models[0]->nodes };
static float time{ 0 };
gltf_models[0]->animate(0/*animation index*/, time += elapsed_time, animated_nodes);
if (gltf_models[0]->animations.at(0/*animation index*/).duration < time)
{
    time = 0; // Repeat playback
}
gltf_models[0]->render(immediate_context.Get(), world, animated_nodes);
```

【評価項目】

☐アニメーションの再生